Agent Skills

前言

大语言模型智能体是近两年研究和工程领域极其火爆的课题，在本系列的上一集 Codex 源码解析当中我们就以 Codex 为例解析了大语言模型智能体的基本实现方式。但在其中仍然缺少了一些重要的内容，也就是关于智能体的扩展性问题。在本篇当中我们就着重补齐这一部分，着重去讲近期被广泛应用的 Agent Skills 机制。

简而言之，Agent Skills 的构建思路很像是 von Neumann 机，也即是当前几乎所有计算机的实现思路。它将程序也作为一种数据进行处理，而这在大语言模型当中的具体实现就是：Agent Skill 当中会封装好使用文本描述的工作流程以及执行任务所需的各种资源，供智能体参考使用。而在具体实现上，Agent Skills 也无非只是一些文本协议以及标准规范。我们这里就做一些简单介绍，并且讲解一下它在 mini-codex 当中的实现情况。更具体的内容可以参考 https://agentskills.io/home，本文主要也参考自官方文档以及 Codex 当中的实现。

什么是 Skills

其实 Skills 的想法和其中所实现的功能早在 GPT-3.5 时代甚至更早就已经普遍存在了，因此它的贡献更多还是在于标准化上面。因为 Skills 统一了这些功能的格式和实现方式，它们就可以被整合和应用进各种不同的智能体当中，以此实现智能体能力的动态扩展。

Skills 所规定的标准格式当中，一个 Skill 就是一个文件夹，其中的大致目录结构如下

```
my-skill/
├── SKILL.md          # Required: instructions + metadata
├── scripts/          # Optional: executable code
├── references/       # Optional: documentation
└── assets/           # Optional: templates, resources
```

其中 SKILL.md 是必须的一个文件，其开头还需要包含一段 yaml 格式的元信息来描述这个 Skill 的名称和它的功能用法 (也可以包含一些其他内容，具体可参见文档)，具体实现起来大致是这个样子

```
---
name: pdf-processing
description: Extract PDF text, fill forms, merge files. Use when handling PDFs.
---

# PDF Processing

## When to use this skill
```

```
Use this skill when the user needs to work with PDF files...

## How to extract text
1. Use pdfplumber for text extraction...

## How to fill forms
...
```

关于 Skills 构建当中的更多标准细节可以参考官方文档 Specification 章节。

在智能体的 system prompt 构建过程当中，首先每一个 skill 都只有 `name` 和 `description` 两个元信息会进入 prompt 当中，而其中更详细的内容则会在实际调用时才会被添加进智能体的上下文。至于正文内容的话，它实际上就是一些预制的 prompt 合集。不过也正因为预制的关系，它实际上可以写得非常详尽并且精心优化，因此根据这些指导执行任务的质量通常都是比较高的。

这里面有一个比较有意思的是 Codex 里面的实现，当我们显式指定智能体去应用一个 skill 并读取其中的内容时，skill 正文其实会被包装成一个 **user message** 添加到对话上下文当中，其格式大概长成这个样子 (参见 codex 源码 `core-skills/src/injection.rs:100`)

```
<skill>
  <name>...</name>
  <path>...</path>
  ...SKILL.md 全文...
</skill>
```

我们大概可以这样去理解为什么 codex 要如此设计 skill 的读取方式：

- Agent 工作当中需要尽可能保持自回归任务当中的 KV Cache 以提升效率，因此读取到的信息都应当插入到对话的末尾
- user message 本身在经典的训练流程当中就被视作「应当要遵循的指令」，因此将 skill 内容作为 user message 插入的话，大模型就能自然的确认并将它作为任务执行。(甚至另一方面，对于 tool 等外源的信息，现代大模型为防范 prompt 注入等攻击专门训练后，可能完全不会遵循其中的指令操作)

当然除去我们显式要求智能体去使用某一个 skill 之外，还有一种可能的情况是我们只给出了一个宽泛的指令，而模型自己根据各个 skill 的描述推断需要使用某一个 skill。实践当中这可能才是更常见的触发类型，此时模型就会隐式地读取 skill 并执行。关于它的具体实现方式，或许直接阅读 skill 相关的 system prompt 去了解最为简单直接。在 Codex 源码当中，这部分内容就被放在 `codex-rs/core-skills/src/render.rs` 这个路径下的 `render_skills_section` 这个函数当中。我们在此也做一些摘录：

```rust
pub fn render_skills_section(skills: &[SkillMetadata]) -> Option<String> {
    if skills.is_empty() {
```

```
        return None;
    }

    let mut lines: Vec<String> = Vec::new();
    lines.push("## Skills".to_string());
    lines.push("A skill is a set of local instructions to follow that is stored
in a `SKILL.md` file. Below is the list of skills that can be used. Each entry
includes a name, description, and file path so you can open the source for full
instructions when using a specific skill.".to_string());
    lines.push("### Available skills".to_string());

    for skill in skills {
        let path_str = skill.path_to_skills_md.to_string_lossy().replace('\\',
"/");
        let name = skill.name.as_str();
        let description = skill.description.as_str();
        lines.push(format!("- {name}: {description} (file: {path_str})"));
    }

    lines.push("### How to use skills".to_string());
    lines.push(
        r###"- Discovery: The list above is the skills available in this session
(name + description + file path). Skill bodies live on disk at the listed paths.
- Trigger rules: If the user names a skill (with `$SkillName` or plain text) OR
the task clearly matches a skill's description shown above, you must use that
skill for that turn. Multiple mentions mean use them all. Do not carry skills
across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read,
say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):
  1) After deciding to use a skill, open its `SKILL.md`. Read only enough to
follow the workflow.
  2) When `SKILL.md` references relative paths (e.g., `scripts/foo.py`), resolve
them relative to the skill directory listed above first, and only consider other
paths if needed.
  3) If `SKILL.md` points to extra folders such as `references/`, load only the
specific files needed for the request; don't bulk-load everything.
  4) If `scripts/` exist, prefer running or patching them instead of retyping
large code blocks.
  5) If `assets/` or templates exist, reuse them instead of recreating from
scratch.
- Coordination and sequencing:
  - If multiple skills apply, choose the minimal set that covers the request and
state the order you'll use them.
  - Announce which skill(s) you're using and why (one short line). If you skip an
obvious skill, say why.
- Context hygiene:
  - Keep context small: summarize long sections instead of pasting them; only
load extra files when needed.
  - Avoid deep reference-chasing: prefer opening only files directly linked from
`SKILL.md` unless you're blocked.
  - When variants exist (frameworks, providers, domains), pick only the relevant
reference file(s) and note that choice.
```

```
- Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue."###
            .to_string(),
    );

    let body = lines.join("\n");
    Some(format!(
        "{SKILLS_INSTRUCTIONS_OPEN_TAG}\n{body}\n{SKILLS_INSTRUCTIONS_CLOSE_TAG}"
    ))
}
```

Codex 所采用的这种实现方式看起来还是比较麻烦的，相比之下我其实更倾向于始终采用单一的隐式触发方案，可能只需数百行代码就能够实现 skills 功能的集成。

Skills 功能的集成

在这一章节我们就来介绍一下 Skills 功能在 mini-codex 当中的具体实现。实际上 Skills 本身只是一些预制的指令以及相关资源，只是具体实现起来仍然会有非常多的麻烦需要处理。这在 Skills 官网指导当中都有详细的介绍，不过我们都不对这些特性做额外处理了。

我们在 mini-codex 当中则按照惯例采用了最简化的实现方式——基本就是纯 prompt 实现。其中最主要的代码都可以参见代码仓库当中的 `src/skills.rs` 文件当中，主要 prompt 也基本上与 codex 保持一致。具体而言在 mini-codex 当中我们主要做了这些事情：

- 从系统以及当前项目目录下的对应位置处读取 Skills 文件
  - 具体而言包括 `~/.agents/skills/`，`~/.mini-codex/skills/`，`./.agents/skills/`，`./.mini-codex/skills/`，以及用户手动通过命令行参数指定的路径。
- 读取并解析每一个 Skill 对应的名称以及描述，记录进 mini-codex 当中
- 采用与 codex 相同的方法合并成一个 skills 章节写入 system prompt 当中
  - 其中包括每一个 skill 的描述以及对 skill 使用方式的整体指导

然后就没有了。对，就是这样简单。如此我们就实现了全隐式的 skill 调用支持。

我们后续可以通过一个具体的第三方 skill 来测试它的工作情况。这里我们选用了金谷园饺子馆 Skill 来用作测试。金谷园是我最喜欢的饺子馆，吃过的都说好！而这个 skill 则可以帮助大模型准确地回答和金谷园饺子馆相关的各种问题，包括在哪吃，几点开门，怎么排队等等。这些信息都是通过 MCP 协议实时获取的，因此实效性非常棒。

要使用这个 skill 也非常简单，我们只需要进入到上述几个 skills 目录之一，然后通过 `git clone` 获取金谷园 skill。接着只需要安装好 mini-codex v0.2.0 (当然使用其他任意一种带有 skills 支持的 agent client 也都可以)，之后就可以实现这样的交互效果了：

```
you> 你好！请问我应该去哪里找到金谷园饺子馆，这家餐馆风格如何，有哪些代表菜式，
wifi密码是多少呢？
assistant>  我先查看金谷园查询技能说明，再给你准确答复。
...
assistant>  我用的是金谷园查询技能。

你可以去这两家店找金谷园饺子馆：
- 北邮店：杏坛路文教产业园K座南2层
- 五道口店：五道口东源大厦4层

这家店的风格我能准确查到的是：
- 简介：北邮旁边的饺子馆
- 整体感觉按店方资料是比较朴实、实在的那种
- 营业时间：10:00-22:00

至于你问的“有哪些代表菜式”，这个我目前还真没法准确说，怕给你说错了。现在我能可靠
获取的信息里还不包含菜单和招牌菜。你可以：
- 直接到店问店员
- 或者在大众点评/美团搜“金谷园饺子馆”看菜单和大家常点的菜

Wi-Fi 信息是：
- Wi-Fi 名称：苹果密码8个8
- 密码：88888888
- 查找方式：开启 Wi-Fi 往底部滑

如果你愿意，我也可以继续帮你整理成“离你最近的一家怎么去”。
```

看起来很不错！当然在此之外的更多 skills 功能理应也能够很好地支持，就有待更多的尝试了。